Computer Organization and Assembly Language (EL-2003)

Lab Mid-Exam

Total Time (mins): 90
Total Marks: 40
Total Questions: 03

Date: Monday, October 05, 2026
Time: 08:20 AM – 10:00 AM

Semester: Fall 2026
Paper: A

Instructor(s):
Mr. Abdullah Shaikh, Mr. Muhammad Owais, Ms. Fareeha Jabeen

Student Name	Roll No	Section	Student Signature

Do not write below this line

Important Instructions:

1. This Paper A consists of two subparts.
2. Subpart A must be attempted on the question paper itself. Write your answers neatly and to the point.
3. A rough sheet is attached for your use.
4. All final answers must be written on the question paper.
5. The time allocation is as follows: Subpart A is allotted 25 minutes, and Subpart B is allotted 65 minutes.
6. In case of any ambiguity, you may make assumptions. However, your assumptions must not contradict any statement in the question paper.
7. Understanding the question is considered part of the examination.

LLO # 02: Understanding and apply assembly language programming concepts, including data definitio initialization, and manipulation, as well as the use of instructions, loops, and built-in procedures to perform compl tasks, and demonstrate an understanding of how to work with arrays, conditional processing, strings, and flags control program flow and perform input/output operations

---

Question # 01 **[Weightage: 05 | Marks: 10 | Time: 25 Min**

Instructions:

1. Attempt all ten parts, (a) to (j), on this question paper in the space provided. Each part carries 01 mar × 1 = 10).
2. Every fragment is placed inside main of a program that includes Irvine32.inc; no other code executes
3. Write outputs exactly as they would appear on the console (including signs, leading zeros and line br
4. In Part B, marks are awarded only when all errors in a fragment are identified and correctly fixed.

Part A – Output Tracing: Write the exact console output produced by each code fragment.

(a) Write the exact output (four lines). **[01]**

```
.data
 wArr WORD 1234h, 0ABCDh,
          5678h
 bArr BYTE 11h, 22h
      BYTE 33h
.code
 mov eax, LENGTHOF wArr
 call WriteDec
 call Crlf
 mov eax, SIZEOF bArr
 call WriteDec
 call Crlf
 mov eax, DWORD PTR [wArr + 1]
 call WriteHex
 call Crlf
 movsx eax, BYTE PTR [wArr + 3]
 call WriteInt
```

Output:

(b) Write the exact output (one line). **[01]**

```
.data
 nums SDWORD 5, -3, 12
.code
 mov ebx, 0
 mov ecx, LENGTHOF nums
L1:
 mov eax, nums[ecx*4 - 4]
 call WriteInt
 mov al, ' '
 call WriteChar
 add ebx, eax
 loop L1
 mov eax, ebx
 call WriteInt
```

Output:

(c) Write the exact output. **[01]**

```
.data
 saveCnt DWORD ?
.code
 mov ecx, 3
mov bl, 'A'
Outer:
 mov saveCnt, ecx
Inner:
 mov al, bl
 call WriteChar
 inc bl
 loop Inner
 call Crlf
 dec bl
 mov ecx, saveCnt
 loop Outer
```

Output:

(d) Write the exact output, and state how many times each label is reached. **[01]**

```
.code
 mov ecx, 3
 mov eax, 1
jmp Mid
Top:
 add eax, eax
 loop Mid
 jmp Done
Mid:
 add eax, 3
loop Top
neg eax
Done:
 call WriteInt
```

Output:

(e) Write the exact output (two lines) and the value of CF after neg ax. **[01]**

```
.data
 d1 DWORD 0F0E0D0C0h
 w1 WORD  8001h
.code
 mov eax, d1
 mov ax, w1
 movsx ebx, ax
 xchg al, ah
 neg ax
 call WriteHex
 call  Crlf
 mov eax, ebx
 call WriteInt
```

Output:

(f) Write the exact output (three lines). **[01]**

```
BASE  = 2 + 3
BASET EQU <2 + 3>
.data
 tbl WORD BASE DUP(BASET * 2)
.code
 mov eax, SIZEOF tbl
 call WriteDec
 call Crlf
 mov eax, BASET * BASET
 call WriteDec
 call Crlf
 movzx eax, tbl[4]
 call WriteDec
```

Output:

Part B – Find & Fix: Each fragment does not do what is intended. Identify every error and write the corrected instruction(s).

(g) ***Intended:*** copy src into dst in reverse order, zero-extended, so dst = 250, 20, 10. **[01]**

```
.data
 src BYTE 10, 20, 250
 dst WORD 3 DUP(?)
.code
 mov esi, OFFSET src + 2
 mov edi, OFFSET dst
 mov ecx, LENGTHOF dst
L1:
 movzx ax, [esi]
 mov [edi], ax
 dec esi
 add edi, 1
 loop L1
```

Error(s) and correction(s):

(h) ***Intended:*** print 3 rows of 4 stars (****). **[01]**

```
.data
 saveC DWORD ?
 star  BYTE "*",0
.code
 mov ecx, 3
Outer:
 mov ecx, 4
Inner:
 mov edx, star
 call WriteString
 loop Inner
 call Crlf
 mov ecx, saveC
 loop Outer
```

Error(s) and correction(s):

(i) ***Intended:*** show a Yes/No box with the message on two lines, then print the button code (6 or 7) in yellow on blue. **[01]**

```
.data
 msg BYTE "Continue?",0,
     0Dh,0Ah,"Yes or No",0
.code
 mov ebx, 0
 mov edx, OFFSET msg
 call MsgBoxAsk
 mov ebx, eax
 mov eax, yellow + (blue * 16)
 call SetTextColor
 call WriteDec
```

Error(s) and correction(s):

(j) ***Intended:*** mat is a 2 × 3 WORD matrix; print each row sum: 6 then 15. **[01]**

```
.data
 mat WORD 1, 2, 3
     WORD 4, 5, 6
.code
 mov ebx, OFFSET mat
mov ecx, LENGTHOF mat
Rows:
 mov edx, ecx
 mov eax, 0
 mov esi, 0
 mov ecx, 3
```

```
; (continued)
Cols:
 add ax, [ebx + esi*TYPE mat]
 inc esi
 loop Cols
 call WriteDec
 call  Crlf
 add ebx, 3
 mov ecx, edx
 loop Rows
```

Error(s) and correction(s):

Part A – Output Tracing: Write the exact console output produced by each code fragment.

(a) Write the exact output (four lines). **[01]**

```
.data
wArr WORD 1234h, 0ABCDh,
          5678h
bArr BYTE 11h, 22h
     BYTE 33h
.code
mov eax, LENGTHOF wArr
call WriteDec
call Crlf
mov eax, SIZEOF bArr
call WriteDec
call Crlf
mov eax, DWORD PTR [wArr + 1]
call WriteHex
call Crlf
movsx eax, BYTE PTR [wArr + 3]
call WriteInt
```

Output:

(b) Write the exact output (one line). **[01]**

```
.data
nums SDWORD 5, -3, 12
.code
mov ebx, 0
mov ecx, LENGTHOF nums
L1:
mov eax, nums[ecx*4 - 4]
call WriteInt
mov al, ' '
call WriteChar
add ebx, eax
loop L1
mov eax, ebx
call WriteInt
```

Output:

(c) Write the exact output. **[01]**

```
.data
saveCnt DWORD ?
.code
mov ecx, 3
mov bl, 'A'
Outer:
mov saveCnt, ecx
Inner:
mov al, bl
call WriteChar
inc bl
loop Inner
call Crlf
dec bl
mov ecx, saveCnt
loop Outer
```

Output:

(d) Write the exact output, and state how many times each label is reached. **[01]**

```
.code
mov ecx, 3
mov eax, 1
jmp Mid
Top:
add eax, eax
loop Mid
jmp Done
Mid:
add eax, 3
loop Top
neg eax
Done:
call WriteInt
```

Output:

(e) Write the exact output (two lines) and the value of CF after neg ax. **[01]**

```
.data
d1 DWORD 0F0E0D0C0h
w1 WORD  8001h
.code
mov eax, d1
mov ax, w1
movsx ebx, ax
xchg al, ah
neg ax
call WriteHex
call Crlf
mov eax, ebx
call WriteInt
```

Output:

(f) Write the exact output (three lines). **[01]**

```
BASE  = 2 + 3
BASET EQU <2 + 3>
.data
tbl WORD BASE DUP(BASET * 2)
.code
mov eax, SIZEOF tbl
call WriteDec
call Crlf
mov eax, BASET * BASET
call WriteDec
call Crlf
movzx eax, tbl[4]
call WriteDec
```

Output:

Part B – Find & Fix: Each fragment does not do what is intended. Identify every error and write the corrected instruction(s).

(g) ***Intended:*** copy src into dst in reverse order, zero-extended, so dst = 250, 20, 10. **[01]**

```
.data
src BYTE 10, 20, 250
dst WORD 3 DUP(?)
.code
mov esi, OFFSET src + 2
mov edi, OFFSET dst
mov ecx, LENGTHOF dst
L1:
movzx ax, [esi]
mov [edi], ax
dec esi
add edi, 1
loop L1
```

Error(s) and correction(s):

(h) ***Intended:*** print 3 rows of 4 stars (****). **[01]**

```
.data
saveC DWORD ?
star  BYTE "*",0
.code
mov ecx, 3
Outer:
mov ecx, 4
Inner:
mov edx, star
call WriteString
loop Inner
call Crlf
mov ecx, saveC
loop Outer
```

Error(s) and correction(s):

(i) ***Intended:*** show a Yes/No box with the message on two lines, then print the button code (6 or 7) in yellow on blue. **[01]**

```
.data
msg BYTE "Continue?",0,
    0Dh,0Ah,"Yes or No",0
.code
mov ebx, 0
mov edx, OFFSET msg
call MsgBoxAsk
mov ebx, eax
mov eax, yellow + (blue * 16)
call SetTextColor
call WriteDec
```

Error(s) and correction(s):

(j) ***Intended:*** mat is a 2 × 3 WORD matrix; print each row sum: 6 then 15. **[01]**

```
.data
mat WORD 1, 2, 3
    WORD 4, 5, 6
.code
mov ebx, OFFSET mat
mov ecx, LENGTHOF mat
Rows:
mov edx, ecx
mov eax, 0
mov esi, 0
mov ecx, 3
; (continued)
Cols:
add ax, [ebx + esi*TYPE mat]
inc esi
loop Cols
call WriteDec
call Crlf
add ebx, 3
mov ecx, edx
loop Rows
```

Error(s) and correction(s):

Important Instructions:

1. This is a coding-based sub-question of Paper A.
2. You are required to attempt Question 02 and Question 03 given in the question paper.
3. After completing your code:
   - Paste your program code in the Word file.
   - Run your program and take output screenshots.
   - Paste the screenshots in the same Word file below your code.
4. Save the file with the naming convention: YourName - YourRollNumber - Paper(A).docx
5. Submit the Word file on the server within the given deadline.
6. Late submissions will not be accepted.

Student Name	Roll No	Section	Student Signature

LLO # 02: Understanding and apply assembly language programming concepts, including data definition, initialization, and manipulation, as well as the use of instructions, loops, and built-in procedures to perform complex tasks, and demonstrate an understanding of how to work with arrays, conditional processing, strings, and flags to control program flow and perform input/output operations

---

Question # 02 **[Weightage: 05 | Marks: 10 | Time: 20 Minutes]**

A lab has four temperature sensors, each with a fixed calibration offset: **−5, 3, −2 and 7**, stored as signed bytes. Write a program that reads the four raw readings from the user and stores them in an array. It then adds each sensor's offset to its reading to produce the adjusted readings. The program must then evaluate:

$$R = -r0 + (r1 - r2) + r3$$

where r0 to r3 are the adjusted readings.

Your program must perform the following: **Marks**

1. Declare the offsets array (signed bytes) and a readings array; read the four raw readings from the user and store them in the array. **[02]**
2. Add each sensor's offset to its reading and display the adjusted readings on one line. **[02]**
3. Evaluate R and display it in decimal and in hexadecimal. **[04]**
4. Display only the low word of R, in hexadecimal. **[01]**
5. Display only the low byte of R, shown as a signed decimal value. **[01]**

Sample Input / Output *(values in bold are typed by the user)*

Sample Run 1

```
Enter reading 1: 25
Enter reading 2: 97
Enter reading 3: -60
Enter reading 4: 42
Adjusted readings : +20 +100 -62 +49
R (decimal)       : +191
R (hexadecimal)   : 000000BF
Low word of R     : 00BF
Low byte of R     : -65
```

Sample Run 2

```
Enter reading 1: 50
Enter reading 2: 10
Enter reading 3: 40
Enter reading 4: -5
Adjusted readings : +45 +13 +38 +2
R (decimal)       : -68
R (hexadecimal)   : FFFFFFBC
Low word of R     : FFBC
Low byte of R     : -68
```

LLO # 02: Understanding and apply assembly language programming concepts, including data definition, initialization, and manipulation, as well as the use of instructions, loops, and built-in procedures to perform complex tasks, and demonstrate an understanding of how to work with arrays, conditional processing, strings, and flags to control program flow and perform input/output operations

[Weightage: 10 | Marks: 20 | Time: 45 Minutes]

Question # 03

A program stores a word as a null-terminated string. Write one program that does all four parts in order.

Rules:

1. Access array elements through registers holding addresses, not by the array name with a fixed offset.
2. Take all loop counts and element sizes from the data declarations; do not hard-code them.

Your program must perform the following: **Marks**

1. Declarations — Declare wordStr, and a DWORD array codes with one element per character of the word (null excluded). Display the element size, element count and total size in bytes of both arrays. **[04]**
2. Staircase — Print the word as a staircase: line 1 shows the first character, line 2 the first two characters, and so on until the full word is shown. **[06]**
3. ASCII codes — Store the ASCII code of each character in codes. Display codes and the sum of all its elements. **[05]**
4. Lowercase in place — Convert the word to lowercase inside the same array, keeping it null-terminated, and display it. **[05]**

Sample Input

The program takes no keyboard input; the word is declared in the data section:

```
wordStr BYTE "ASSEMBLY",0
```

Sample Output

```
(a) Array properties   ElementSize ElementCount TotalSize
    wordStr            1           9            9
    codes              4           8            32

(b) Staircase
    A
    AS
    ASS
    ASSE
    ASSEM
    ASSEMB
    ASSEMBL
    ASSEMBLY
```

```
(c) Codes : 65 83 83 69 77 66 76 89
    Sum   : 608

(d) Lowercase : assembly
```

Return the question paper after completion

Wishing you success in your exam!